

AWS Cost Detective

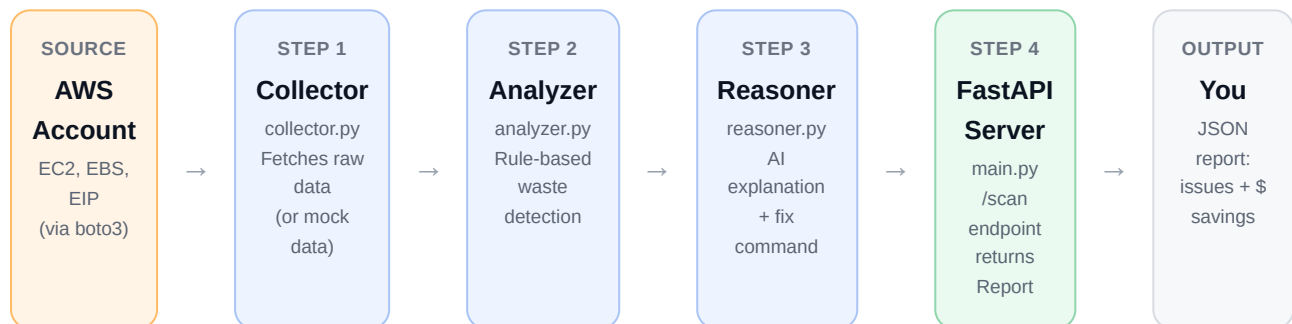
An AI-assisted tool that scans an AWS account, finds wasted spend, and explains the fix in plain English.

By **Suchith Jain** · August 2, 2026

What it does

AWS Cost Detective connects (read-only) to an AWS account, inspects EC2 instances, EBS volumes, and Elastic IPs, and flags resources that are quietly costing money — an oversized server, an orphaned disk, an unattached IP address. For each issue it estimates the monthly dollar cost and produces the exact AWS CLI command to fix it. The result is returned as a JSON report via a small FastAPI web service (and can be run standalone from the terminal using mock data).

How It Works — the assembly line



Why the AI is boxed into one step: problem *detection* is done entirely by deterministic rules in `analyzer.py` — never by the AI. The OpenAI model in `reasoner.py` only turns an already-confirmed finding into a friendlier explanation. This guarantees the AI can never invent a fake issue or a wrong number; the rules stay authoritative and the AI just narrates.

Supporting modules

FILE	ROLE
<code>models.py</code>	Pydantic schemas that define the shape of findings, fixes, and the final report.
<code>pricing.py</code>	Rough US-East price table used to convert a finding into a monthly dollar estimate.
<code>demo.py</code>	Runs the full pipeline in the terminal against mock data — zero AWS setup required.

What It Detects

1 Over-provisioned servers

EC2 instances far bigger than their actual usage requires.

2 Unused disks

EBS volumes attached to nothing but still billed.

3 Old disk type (gp2)

Volumes on gp2 that could move to cheaper, faster gp3.

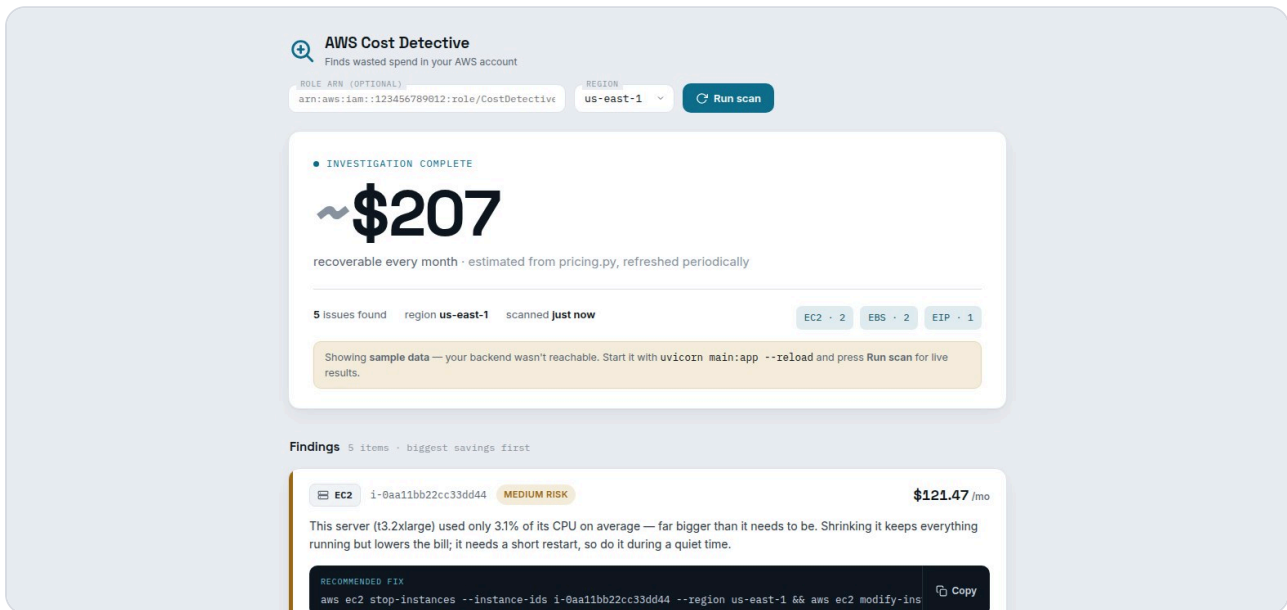
4 Idle IP addresses

Elastic IPs reserved but not attached to any resource.

New checks are simple rules added to `analyzer.py`, so the detector list is easy to extend.

Live app — example result

The tool ships with a small web frontend (`frontend/index.html`) that calls the `/scan` endpoint and renders the report as cards. Below is the app running against its built-in sample data (shown automatically whenever the backend isn't reachable, so the demo works offline).



Dollar figures are estimates from the price table in `pricing.py` — refresh it periodically to keep numbers current.

Tech stack

CLOUD DATA
boto3

BACKEND
Python + FastAPI

AI
OpenAI (optional)

VALIDATION
Pydantic

FRONTEND
HTML (no build)

Prerequisites

REQUIREMENT	NOTES
Python 3.10+	Runs the FastAPI backend and the pipeline scripts.
<code>pip install -r requirements.txt</code>	Installs fastapi, uvicorn, boto3, pydantic, python-dotenv, and openai (optional).
AWS account	Optional for trying the tool — mock mode needs none. Required only for scanning real resources.
Read-only AWS IAM key	Only needed when scanning a real account (<code>USE MOCK DATA=false</code>). Use the built-in <code>ReadOnlyAccess</code> policy — the tool never writes or deletes anything.
OpenAI API key	Optional. Only used to make explanations friendlier; the tool works fully without one.

How to Run

MODE	COMMAND	NOTES
Terminal demo	<code>python demo.py</code>	Mock data, no AWS account needed.
Web server	<code>uvicorn main:app --reload</code>	Then call <code>/scan?region=us-east-1</code> .
Real AWS scan	set <code>USE MOCK DATA=false</code> in <code>.env</code>	Use a read-only IAM key (ReadOnlyAccess policy). The tool never modifies resources.

AWS Cost Detective — architecture & project overview · Suchith Jain · August 2, 2026

The scan covers one AWS region at a time, set via the `?region=` option on `/scan` (defaults to `us-east-1` if omitted). Reading resource metadata is read-only and stays within AWS's normal API rate limits.